INSTITUTE OF ENGINEERING & MANAGEMENT, KOLKATA

COURSE TITLE: BACHELOR OF TECHNOLOGY

SUBJECT: PYTHON PROGRAMMING LABRORATORY

PROJECT TITLE: WATER CONSUMPTION AND LEAK ALERT ANALYSER

NAME:

1. SIDDHARTHA SAHA
2. JYOTIRMOY HALDER
3. MAYUKH BHATTACHARJEE
4. VIR SUTIRTHA CHANDA
5. ANKAN SARKAR

ROLL NO:

1. 237
2. 238
3. 240
4. 241
5. 244

ENROLLMENT NO:

1. 12025052003255
2. 12025052003256
3. 12025052003258
4. 12025052003259
5. 12025052003262

SECTION : C

SUBMITTED TO : SOHAM GHOSH SIR AND IB SIR

DEPARTMENT NAME: ELECTRONICS AND COMMUNICATION ENGINNERING

ACADEMIC SESSION: 2026-2027

DATE OF SUBMISSION: 12-11-2026

WATER CONSUMPTION AND LEAK-ALERT ANALYSER

Abstract

This project presents a Python-based campus water-consumption monitoring and leak-alert analysis pipeline. It processes cumulative meter readings from 11 campus buildings over 120 days (1,320 meter-days), handles missing telemetry and non-monotonic meter readings, estimates daily consumption using a gap-aware differential method, and evaluates six leak-detection strategies against labelled ground-truth leak events. The software uses a WaterMeter object-oriented data model and NumPy, Pandas and Matplotlib for numerical processing, reporting and visualisation. It exports building-level and monthly reports in CSV/JSON formats and generates 14 analytical plots. The submitted project reports 16 passing tests. Among the evaluated strategies, Rule D reports 1.00 precision and 0.62 recall. The demonstration uses simulated telemetry (seed 42), so the results demonstrate algorithm behaviour on the supplied synthetic scenario and do not establish field performance in a live campus network.

Keywords

Water consumption; leak detection; smart metering; telemetry; missing data; gap-aware rate; anomaly detection; Python; NumPy; Pandas; Matplotlib; precision; recall.

Table of Contents

Section	Topic
1.	Introduction
2.	Problem Statement and Objectives
3.	Software and System Requirements
4.	System Design and Methodology
5.	Implementation of Major Modules
6.	Results and Discussion
7.	Testing and Validation
8.	Limitations and Future Scope
9.	Conclusion
10.	References
11.	Appendix: Data Files and Execution

Project basis: the supplied Water-Consumption & Leak-Alert Analyser project document and its embedded charts. All numerical findings below are reported as shown in that source; unreported values are not estimated.

1. Introduction

Water distribution systems in institutional buildings can experience continuous plumbing leaks, underground pipe damage, missing sensor transmissions and meter-counter resets. These conditions complicate the interpretation of smart-meter data. A simple difference between consecutive readings can create misleading consumption values when readings are missing or when a meter counter decreases unexpectedly.

The Water-Consumption & Leak-Alert Analyzer is designed as an end-to-end software pipeline for campus telemetry. It ingests cumulative meter readings, checks data quality, computes daily consumption rates, estimates building baselines, evaluates leak-detection rules and produces reports and diagnostic plots.

2. Problem Statement and Objectives

Problem statement: Develop a reliable analytical system that identifies abnormal water-consumption patterns across multiple campus buildings while avoiding false alarms caused by communication gaps, sensor noise and non-monotonic meter readings. Compare multiple detection strategies using labelled ground-truth leak events.

The principal objectives are:

- Simulate and process cumulative meter telemetry for 11 buildings over 120 days.
- Detect missing readings and counter rollbacks before calculating consumption.
- Replace misleading carry-forward differences with gap-aware daily rates.
- Encapsulate meter data and operations in a reusable WaterMeter class.
- Compare six leak-detection rules using precision, recall and false-alarm behaviour.
- Generate building-level, monthly and rule-evaluation reports in CSV/JSON formats.
- Produce plots that support campus-wide and per-building diagnostics.

3. Software and System Requirements

Component / Requirement	Purpose in the project
Python 3.9+	Core language and execution environment.
NumPy	Array operations, missing-value masks, rates and numerical calculations.
Pandas	Tabular data processing, aggregation and CSV/JSON reporting.
Matplotlib	Generation of campus overview, rule comparison and building diagnostic plots.
CSV / JSON files	Input telemetry, ground-truth labels, benchmark results and exported summaries.
Python test runner / pytest	Execution of the automated validation suite.

4. System Design and Methodology

4.1 Input data and monitoring scope

The project describes a synthetic dataset containing 11 facilities and 120 days of observations, giving 1,320 meter-days. Each building has a cumulative meter-reading time series. The project also includes ground-truth leak labels and a missing-data sweep used to evaluate detection performance. A fixed random seed of 42 is identified for the synthetic data generation.

4.2 Data preparation and quality checks

The ingestion stage identifies NaN readings and non-monotonic decreases. Missing readings are not treated as zero-consumption days. Counter rollbacks are flagged so negative differences do not contaminate consumption rates or baseline statistics. The source describes rollbacks as possible consequences of power interruption, register overflow, meter replacement or sensor reset.

4.3 Processing Pipeline

The proposed system follows a systematic processing pipeline to analyse water consumption data, identify irregularities, detect potential leakage patterns, and evaluate the performance of different detection rules. The complete workflow consists of the following stages:

Stage	Operation	Description
1. Telemetry Input	Data Acquisition	Reads or generates cumulative water meter readings for the monitored buildings.
2. Data Quality Checks	Data Validation	Identifies missing (NaN) values and non-monotonic counter readings to ensure data reliability.
3. WaterMeter Model	Data Representation	Creates an individual WaterMeter object for each building and maintains its corresponding time-series data.
4. Consumption Engine	Consumption Calculation	Calculates gap-aware daily consumption rates and records elapsed time intervals between valid readings.
5. Baseline Estimation	Statistical Analysis	Computes a short-window moving mean and a trailing 28-day median to establish normal consumption patterns.
6. Rule Evaluation	Leak Detection	Evaluates Rules A0, A, and C against the available ground-truth data to identify potential water leakage.
7. Performance Benchmarking	Accuracy Assessment	Generates confusion matrices and calculates performance metrics to measure the effectiveness of the detection rules.
8. Reporting and Visualization	Result Generation	Generates CSV and JSON summary reports and produces graphical visualizations using Matplotlib.

4.4 Mathematical Formulations

Mathematical formulations Cumulative consumption: For consecutive valid meter readings, the ordinary daily difference is $\Delta C_t = C_t - C_{t-1}$. This is suitable only when the readings are valid and consecutive. Gap-aware rate: If valid readings occur at indices a and b with $n = b - a$ elapsed days, the average daily rate is calculated as $(C_b - C_a)/(b - a)$. The elapsed interval is retained as the gap length. This distributes the measured volume over the missed interval rather than creating a zero-consumption period followed by a catch-up spike. Non-monotonic check: A valid reading is flagged when it is lower than the previous valid reading. This prevents a counter rollback from being interpreted as negative water use. Baseline and persistence: The source compares moving-mean baselines with a trailing 28-day median. Rule D uses a median baseline and a two-day persistence gate requiring consumption to remain above 1.20 times the baseline.

5. Implementation of Major Modules

The system is implemented using a modular programming approach, where each module performs a specific function related to water meter data management, consumption analysis, anomaly detection, and result visualization.

The major modules and their respective responsibilities are described below.

Module / Function	Description and Responsibility
WaterMeter	Encapsulates building-wise water meter readings, time indices, and time-series operations.
len() / repr()	Provides the total number of records and a concise textual representation of the WaterMeter object.
getitem()	Supports slicing operations, such as meter[30:60], to extract a specific time window as a new WaterMeter object.
iter()	Enables iteration through the meter's date and corresponding reading pairs.
consumption()	Calculates gap-aware daily consumption rates and provides information about elapsed intervals between valid readings.
non_monotonic()	Identifies and flags meter readings that decrease compared with the previous valid reading.
moving_mean()	Calculates the trailing moving average of water consumption, using a default window size of seven days.
trailing_median()	Computes the trailing median consumption baseline, which is used by robust anomaly-detection rules.
rule_optimal()	Implements the selected Rule D configuration for identifying abnormal water consumption patterns.
Reporting and Plotting Scripts	Generate building-wise reports, monthly summaries, rule evaluation results, and graphical visualizations for performance analysis.

5.1 Main Program Files

The implementation primarily consists of the following Python source files:
1. water_analyser.py – Contains the core water consumption analysis logic, including the WaterMeter class, consumption calculations, statistical baselines, and detection rules.
2. make_plots (1).py – Responsible for generating graphical representations, statistical plots, and visual reports of the analysed water consumption data.
3. test_water_analyser.py – Contains test cases designed to verify the functionality and correctness of the water analysis modules.

5.2 Dataset and Output Files

In addition to the main Python programs, the project includes several CSV and JSON files used for data storage, experimentation, evaluation, and reporting.

These datasets include:

- Meter Readings Dataset: Stores cumulative water meter readings collected from different buildings.
- Ground-Truth Leak Dataset: Contains reference information about known leakage events for validating detection results.
- Building Summary Dataset: Provides building-wise water consumption statistics and analysis results.
- Monthly Summary Dataset: Stores monthly consumption patterns and aggregated statistical information.
- Rule Evaluation Dataset: Contains the results obtained from evaluating different leak-detection rules.
- Missing-Data Experiment Dataset: Supports experiments involving missing meter readings and evaluates the system's ability to handle incomplete data.

Overall, the modular implementation ensures that data acquisition, preprocessing, consumption calculation, anomaly detection, performance evaluation, and visualization are handled independently. This improves the maintainability, reliability, and extensibility of the water consumption monitoring system.

6. Results and Discussion

6.1 Evaluation scope and reported headline results

Indicator	Reported value
Monitored buildings	11
Monitoring period	120 days
Total meter-days	1,320
Detection strategies compared	6 (A0, A, A', B, C and D)
Automated tests	16 passed; 0 failed; 0 reported errors
Rule D precision	1.00 (100%)
Rule D recall	0.62 (62%)
Analytical plots generated	14

These values describe the submitted synthetic evaluation. Precision of 1.00 means the reported Rule D predictions contain no false positives in the evaluated set; recall of 0.62 indicates that the rule identifies 62% of the ground-truth leak cases under that evaluation. High precision does not mean that all leaks are detected.

6.2 Campus-wide consumption overview

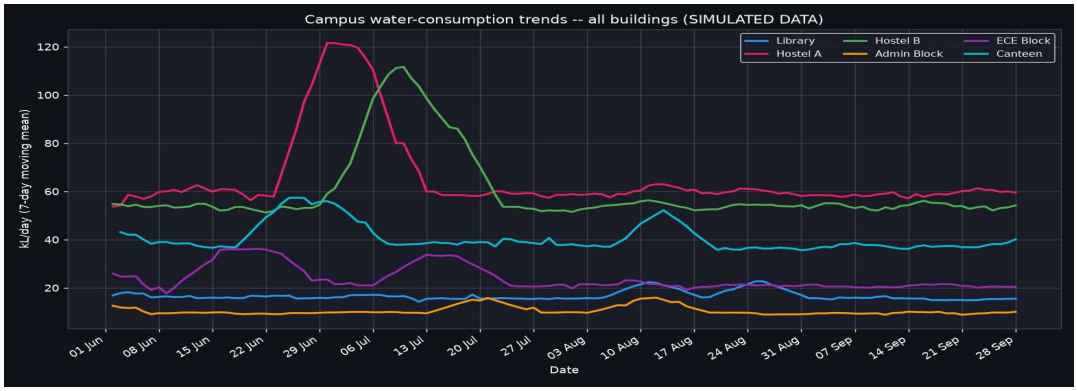


Figure 1. Campus-wide seven-day rolling-average consumption trends for the monitored buildings (simulated data).

The overview compares the time-varying consumption profiles of the campus facilities. Distinct peaks appear in some building traces, while other profiles remain comparatively stable. These patterns help identify periods and buildings that merit closer inspection; the chart alone does not prove that a peak is a leak.

6.3 Comparison of leak-detection rules

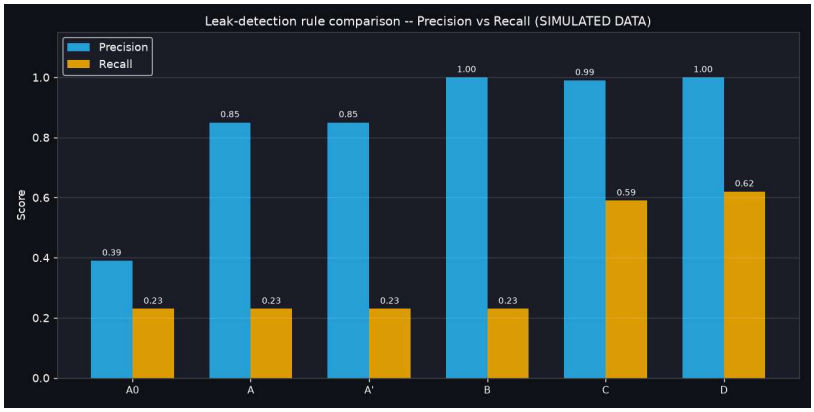


Figure 2. Precision and recall comparison across the six detection strategies (simulated data).

The plotted values are: A0 (0.39 precision, 0.23 recall), A (0.85, 0.23), A' (0.85, 0.23), B (1.00, 0.23), C (0.99, 0.59) and D (1.00, 0.62). Rule D has the highest displayed recall while retaining perfect displayed precision. Rule C is close, with 0.99 precision and 0.59 recall.

6.4 Robustness to missing telemetry

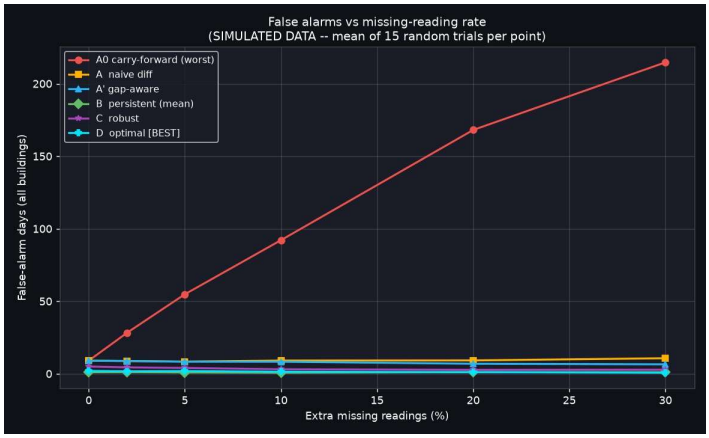


Figure 3. False-alarm days as the extra missing-reading rate increases (simulated data).

The source provides exact Rule A0 false-alarm values of 9.0, 28.3, 54.8, 92.1, 168.1 and 214.7 days at missing rates of 0%, 2%, 5%, 10%, 20% and 30%, respectively. The plotted robust strategies remain close to the bottom of the graph. Exact numeric values for those other curves are not supplied in the document, so they are not reproduced as a table.

6.5 Per-building diagnostic examples

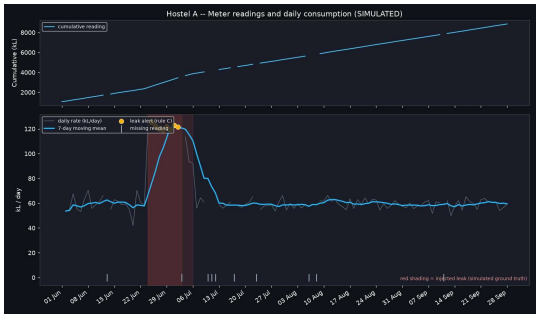


Figure 4. Hostel A diagnostic trace: cumulative meter readings and daily-rate analysis.

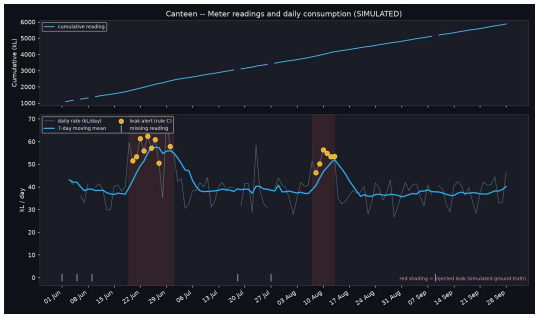


Figure 5. Canteen diagnostic trace: cumulative meter readings and daily-rate analysis.

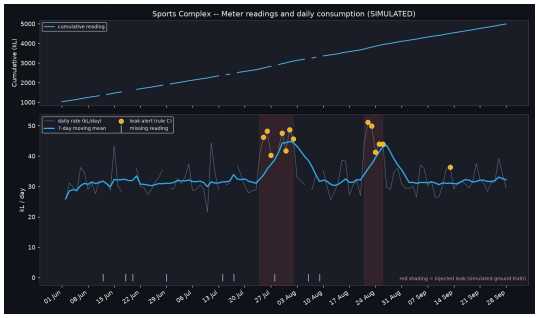


Figure 6. Sports Complex diagnostic trace: cumulative meter readings and daily-rate analysis.

The project also displays an ECE Block diagnostic trace in its visual gallery. These diagnostic plots combine cumulative readings with daily rates, moving averages, ground-truth leak intervals and detector flags. They are examples selected from the project's larger gallery; the source states that 14 plots are generated in total.

6.6 Additional diagnostic example

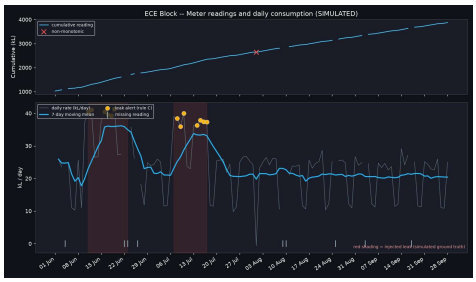


Figure 7. ECE Block diagnostic trace, including a marked non-monotonic reading.

6.7 Campus building profiles

Building	Profile type	Base (kL/day)
Library	Academic	18
Hostel A	Residential	60
Hostel B	Residential	55
Admin Block	Administrative	12
ECE Block	Academic / Labs	25
Canteen	Commercial / Dining	40
Hostel C	Residential	58
Sports Complex	Athletic Facility	30
Lab Block	Research Laboratories	22
Staff Quarters	Residential	35
Auditorium	Event Venue	8

7 Testing and Validation

The submitted test-suite output reports 16 passed tests and no failures or errors. The listed coverage includes special-method behaviour, gap-aware consumption, carry-forward edge cases, moving calculations, non-monotonic readings, rule evaluation, score/metric edge cases and building-report generation. The documented commands are `python test_water_analyser.py` and `pytest -v test_water_analyser.py`. The test result is based on the output shown in the supplied project document.

8 Limitations and Future Scope

The project's evaluation is based on synthetic data generated with seed 42, not on live campus meter measurements. The source does not provide a field deployment, measured water-saving result, operating cost, or validation across different meter hardware. Gap-aware division spreads the volume evenly over a missing interval; this is an average-rate estimate and cannot reveal the exact day on which water was used. A two-day persistence rule can also miss short-duration leaks. The displayed precision and recall therefore should not be treated as guaranteed real-world performance.

Future work could include deployment on consented live meter feeds, real-time alert delivery, configurable building baselines, calibration against actual maintenance-confirmed leaks, dashboard-based monitoring, and evaluation across longer periods and different meter types. These are suggested extensions, not features established as implemented in the supplied source.

9. Conclusion

The Water-Consumption & Leak-Alert Analyzer demonstrates a structured approach to campus water telemetry analysis. Its main engineering contributions are gap-aware consumption-rate calculation, non-monotonic reading checks, a reusable WaterMeter class, comparison of six detection rules, and automated reporting and visualization. On the supplied simulated evaluation, Rule D achieves 1.00 precision and 0.62 recall, and the test suite reports 16 passing tests. The results support the value of robust preprocessing and persistent anomaly checks, while further validation with real meter data is required before operational deployment.

10. References

1. Supplied Project Repository: Water-Consumption & Leak-Alert Analyzer – Project README documentation, source code, datasets, and associated file descriptions.

2. Python Software Foundation. Python 3 Documentation. Available at: https://docs.python.org/3/

3. NumPy Developers. NumPy Official Documentation. Available at: https://numpy.org/doc/

4. pandas Development Team. pandas Official Documentation. Available at: https://pandas.pydata.org/docs/

5. Matplotlib Development Team. Matplotlib Official Documentation. Available at: https://matplotlib.org/stable/

11. Appendix: Data Files and Execution

Files listed in the supplied project: water_analyser.py; make_plots (1).py; test_water_analyser.py; meter_readings.csv; building_report.csv; building_report.json; monthly_report.csv; rule_evaluation.csv; missing_sweep.csv; ground_truth_leaks.csv; ground_truth_leaks.json; and generated plot files.

Execution: Install Python 3.9+ and the listed dependencies (NumPy, Pandas and Matplotlib). Run `python water_analyser.py` for the full data-generation, cleaning, benchmarking, reporting and plotting pipeline. Run `python "make_plots (1).py"` to regenerate plots from existing CSV data. Run the test script or pytest command described in Section 7 to execute validation.